Chapter 1: Binary Search Trees & Tree Traversal

A Binary Search Tree (BST) is a node-based binary tree data structure with specific properties.
The left subtree of a node contains only nodes with keys lesser than the node's key.
The right subtree of a node contains only nodes with keys greater than the node's key.
Both the left and right subtrees must also be binary search trees.

Time Complexity for BST operations:
- Search: Average case is O(log n), but worst case is O(n) when the tree is unbalanced (skewed).
- Insertion: Average case is O(log n), worst case is O(n).
- Deletion: Average case is O(log n), worst case is O(n).

Tree traversals include In-order, Pre-order, and Post-order traversal.
In-order traversal of a binary search tree visits nodes in ascending sorted numerical order.
Balanced variants such as AVL trees and Red-Black trees guarantee O(log n) worst-case time.

Chapter 2: Recursion and Divide & Conquer

Recursion is a problem-solving technique where a function calls itself to solve smaller subproblems.
Every recursive algorithm must contain two fundamental components:
1. Base Case: The condition under which the function terminates without calling itself.
2. Recursive Step: The rule that reduces the problem towards the base case.

Without a proper base case, recursion results in infinite calls and a stack overflow error.

Divide and Conquer Algorithms:
This paradigm divides the problem into sub-problems, solves them recursively, and combines results.
Classic examples include Merge Sort and Quick Sort.
Merge Sort has a guaranteed time complexity of O(n log n) and requires O(n) auxiliary space.
Call stack memory consumption is directly proportional to the maximum recursion depth.

Chapter 3: Dynamic Programming & Memoization

Dynamic Programming (DP) is an optimization technique used for problems with overlapping subproblems and optimal substructure properties.

Difference between Plain Recursion and Dynamic Programming:
Plain recursion recomputes solutions to identical subproblems repeatedly, causing exponential time complexity O(2^n), as observed in the naive Fibonacci sequence.
Dynamic programming saves solutions to already solved subproblems in memory to avoid redundant work.

Two main implementation approaches:
1. Top-Down Approach (Memoization): Uses recursion but stores results in a lookup table (cache).
2. Bottom-Up Approach (Tabulation): Solves smallest subproblems first iteratively and builds up.

Dynamic programming reduces exponential time algorithms to polynomial time, typically O(n) or O(n^2).